



PREDator

Prediction & Forecast Engine

DEVELOPER GUIDE

Architecture, codebase, and extension reference.

PROJECT STRUCTURE

```
PREDator/
├── gui.py           ← Tkinter frontend
├── main.py          ← Entry point, config loader
├── config.yaml      ← Runtime configuration
├── src/
│   ├── pipeline.py  ← Main pipeline orchestrator
│   ├── train.py     ← Keras training loop + callback
│   ├── evaluate.py   ← Metrics (RMSE, MAE, R²) + plots
│   ├── models.py    ← RNN architecture definition
│   ├── gpr_module.py ← GPR model wrapper
│   ├── preprocessing.py ← Data cleaning / dataimport
│   └── utils.py      ← resolve_path, ensure_dir, logging, sequences
```

EXECUTION FLOW

gui.py → main.run_main_pipeline() → pipeline.run_pipeline() → train.train() or GPR.fit_and_predict() → evaluate.final_test()

CONFIGURATION — CONFIG.YAML

All runtime parameters live in config.yaml. The GUI reads and writes this file at every run. Key fields not exposed in the GUI:

- model.n_neurons — number of RNN hidden units. Edit manually in config.yaml.
- test.interval_epochs — how often the TestEveryNEpochs callback fires during training.
- paths.* — all output directories, resolved relative to the executable via resolve_path().

ADDING A NEW RNN ARCHITECTURE

PREDator is model-agnostic. To integrate a new architecture:

1. Define your model class in src/models.py (or a new file in src/).
2. In src/pipeline.py, replace the single instantiation line:

```
rnn = RNN(x_train, y_train, n_epochs, forecast_horizon, n_neurons)
```
3. Training, evaluation, and output saving are handled automatically by the rest of the pipeline.

KEY IMPLEMENTATION NOTES

- Threading — the pipeline runs in a daemon thread via Python's threading module to keep the GUI responsive.
- Stdout/stderr — both are redirected to the GUI console via TextRedirector with ANSI escape code stripping.
- Keras verbosity — verbose=0 in train.py suppresses raw epoch logs; the TestEveryNEpochs callback handles per-epoch output instead.
- Metrics — $RMSE = \sqrt{MSE}$. $NRMSE = RMSE / \text{mean}(I_{y_true})$. Both are saved to rmse.txt after each run.
- Path resolution — all paths go through resolve_path() in utils.py, which uses os.path.dirname(sys.executable) when compiled and os.getcwd() in development.
- GPR folder — gpr/ is only created when rnn_flag=0. Remove the ensure_dir(gpr_dir) call from generate_paths() to enforce this.

DEBUG MODE

At the top of gui.py:

```
DEBUG = 0    # set to 1 to enable full traceback in console
```

When DEBUG=1, any exception in the pipeline thread prints the full traceback to the GUI console. Set back to 0 before compiling for production.

COMPILING WITH PYINSTALLER

```
pyinstaller --onedir --windowed --name PREDator --icon icona.icns \  
--add-data 'config.yaml:.' --add-data 'src:src' gui.py
```

After compiling, manually copy config.yaml into dist/ next to PREDator.app. The app reads and writes config from that location at runtime.

RUNNING IN DEVELOPMENT

```
cd /path/to/PREDator  
source venv/bin/activate  
python gui.py
```

DEPENDENCIES

- Frontend — tkinter, matplotlib (Agg backend).
- ML — TensorFlow / Keras, scikit-learn (MinMaxScaler, mean_squared_error).
- Data — pandas, numpy.
- Config — PyYAML.
- Packaging — PyInstaller.

REPOSITORY

Full codebase: <https://github.com/antoniomazzitelli/PREDator.git>
Designed by Antonio Mazzitelli.